

The StarForth Cookbook

Patterns, Recipes, and Idioms for FORTH Programmers

Robert A. James

Contents

I	FORTH Fundamentals	1
1	Stack Idioms	3
2	Defining Words and Vocabularies	5
3	Factoring Recipes	7
II	Working with Blocks	9
4	Block File Layout	11
5	Block I/O Recipes	13
6	Capsule Programming	15
6.1	Mama Forth: The Multi-VM Supervisor	15
6.1.1	VM-Level Physics Metadata	15
6.1.2	Block Device Physics Metadata	16
6.1.3	Mama’s Responsibilities	16
6.1.4	The Supervision Loop	16
6.1.5	Three-Tier Feedback Loop	17
6.1.6	Worked Example: Thermal Emergency	17
6.1.7	Implementation Phases	17
III	Physics Runtime Recipes	19
7	Understanding Execution Heat	21
8	Tuning the Rolling Window	23
8.1	Instrumentation Gaps and Tuning Knobs	23
8.1.1	Dead Code	23
8.1.2	Stack Metrics (Zero Collection)	23
8.1.3	Dictionary Metrics (Zero Visibility)	23
8.1.4	Block I/O Metrics (Completely Dark)	24
8.1.5	Per-Word Execution Metrics	24
8.1.6	Heat Dynamics (Phase 2)	24
8.1.7	Pipelining Metrics (Instrumented, Unused)	24
8.1.8	Cache Pollution and Cycles	24
8.1.9	Priorities	24

9	Pipelining and Prefetch	27
9.1	Context Window and Binary-Chop Tuning	27
9.1.1	The Trade-Off	27
9.1.2	Knob #6: TRANSITION_WINDOW_SIZE	27
9.1.3	Binary-Chop Search	27
9.1.4	Expected Outcome	28
9.1.5	Interaction With Other Knobs	28
9.1.6	Status	28
10	Reading Heartbeat Output	29
10.1	Heartbeat CSV Export	29
10.1.1	CSV Format	29
10.1.2	Capturing Metrics	29
10.1.3	Analysis Use Cases	30
10.1.4	Implementation	30
10.1.5	Known Limitations	31
11	Physics Freeze and Thaw	33
IV	Design of Experiments	35
12	The 2⁷ Factorial Design	37
13	Running the Factorial Harness	39
13.1	Quick Start: The 2 ⁶ Factorial DoE	39
13.1.1	Three Run Sizes	39
13.1.2	What Is Tested	39
13.1.3	Generated Artifacts	40
13.1.4	Execution Behavior	40
13.1.5	Analysis	40
13.2	Complete 2 ⁶ Factorial Design of Experiments	40
13.2.1	Design Rationale	40
13.2.2	The Six Feedback Loops	40
13.2.3	Configuration Naming	40
13.2.4	Execution	41
13.2.5	Output Structure	41
13.2.6	Analysis	41
13.2.7	Key Design Principles	42
13.3	Factorial DoE Analysis Workflow	42
13.3.1	End-to-End Process	42
13.3.2	Phase 1: Data Collection	42
13.3.3	Phase 2: Data Loading	43
13.3.4	Phase 3: Main Effects	43
13.3.5	Phase 4: Two-Way Interactions	43
13.3.6	Phase 5: Optimal Configuration Search	44
13.3.7	Phase 6: Reporting	44
14	Heartbeat DoE	45
14.1	Quick Start: The Heartbeat DoE	45
14.1.1	Running the Experiment	45
14.1.2	The Five Configurations	45
14.1.3	Analysis	45

14.1.4	Key Metrics	46
14.1.5	Troubleshooting	46
14.1.6	Where This Fits	46
14.2	Heartbeat DoE Execution Guide	46
14.2.1	Overview	46
14.2.2	Pre-Execution Checklist	46
14.2.3	Execution	47
14.2.4	Output Verification	47
14.2.5	Analysis	47
14.2.6	Metric Interpretation	47
14.2.7	Troubleshooting	48
15	Optimization DoE	49
15.1	Quick Start: The Optimization DoE	49
15.1.1	The Five Opportunities	49
15.1.2	Picking a Winner	49
15.1.3	Reading Q _{48.16} Metrics	49
15.1.4	Recommended Sequencing	50
15.1.5	Troubleshooting	50
15.2	Physics Optimization DoE Guide	50
15.2.1	Overview	50
15.2.2	Q _{48.16} Fixed-Point Format	50
15.2.3	Optimisation Opportunities	50
15.2.4	Execution	51
15.2.5	Analysis	51
15.2.6	Expected Cumulative Improvement	51
15.3	Physics Engine Validation — Execution Guide	51
15.3.1	Overview	51
15.3.2	Execution	52
15.3.3	Output Schema	52
15.3.4	Success Criteria	52
15.3.5	System Stability	52
15.3.6	Publication Notes	53
V	Security and ACL Recipes	55
16	Word-Level ACL Basics	57
16.1	Word-Level ACL System	57
16.1.1	Three Enforcement Dispositions	57
16.1.2	Statistically Adaptive TTL	57
16.1.3	The Pin Ratchet	57
16.1.4	Inheritance at Birth	58
16.1.5	Interpreter Hook: Two-Level Check	58
16.1.6	Two-Console Architecture	58
16.1.7	Superuser: Zuse	58
16.1.8	CA Root	59
16.1.9	Security Model and No-Security Condition	59
16.1.10	Self-Activating Capsule	59
16.1.11	ACL _{4th} : Pure-FORTH Implementation	59
16.1.12	Bootstrap Sequence	60
16.1.13	Capsule Namespace	60

16.1.14 Implementation Status	60
16.1.15 Remaining Work	61
17 The Zuse Superuser Console	63
18 Emergency Console Fallback	65
A Build Flags Quick Reference	67
A.1 Build Options Reference	67
A.1.1 Performance Optimization Options	67
A.1.2 Architecture Options	68
A.1.3 Platform / Target Options	68
A.1.4 Debugging and Profiling Options	69
A.1.5 Block System Options	69
A.1.6 Memory Configuration Options	69
A.1.7 Feature Flags	70
A.1.8 Build System Variables	70
A.1.9 Build Configuration Matrix	70
A.1.10 Optimization Level Guidelines	70
A.1.11 Usage Examples	71
A.1.12 Compiler Support	71
A.1.13 Performance Tuning Checklist	72
A.1.14 Troubleshooting	72
B HOLA Analytics Protocol	73
B.1 The HOLA Shared-Memory Protocol (Phase 1)	73
B.1.1 Memory Layout	73
B.1.2 Event Records	73
B.1.3 Channels	74
B.1.4 Command Protocol	74
B.1.5 Synchronization Rules	74
B.1.6 Artefacts	75
C Physics Design Notes	77
C.1 Physics Signal Inventory	77
C.1.1 L4Re and Microkernel Signals	77
C.1.2 StarForth VM Signals	78
C.1.3 Op-Amp Signal Flow	78
C.1.4 Messaging and IPC Considerations	78
C.1.5 Host Snapshot Shim and Analytics Heap	79
C.1.6 Conventions and Constraints	79
C.1.7 Immediate Research Tasks	79

Part I

FORTH Fundamentals

Chapter 1

Stack Idioms

Chapter 2

Defining Words and Vocabularies

Chapter 3

Factoring Recipes

Part II

Working with Blocks

Chapter 4

Block File Layout

Chapter 5

Block I/O Recipes

Chapter 6

Capsule Programming

6.1 Mama Forth: The Multi-VM Supervisor

Mama Forth is the governing FORTH VM that orchestrates a multi-VM ecosystem. It departs from conventional supervisor design in one decisive respect: Mama Forth is itself a FORTH VM, a first-class citizen carrying the same physics instrumentation as every VM it governs. The supervisor does not stand outside the runtime model; it participates in it.

The ecosystem forms a tree. Mama Forth sits at the root. Beneath it, child VMs each carry their own physics state. Beneath each child, block devices carry per-device physics. Mama observes all three layers and makes system-level decisions: when to spawn or kill a child VM, how to allocate block devices to children, how to enforce governance across the ecosystem, and how to respond to emergencies such as thermal runaway or memory pressure.

The central design insight is that physics metadata becomes a three-tier hierarchy:

- **Word level** — per-word metrics: entropy, temperature, latency.
- **VM level** — per-VM aggregates: child health, resource usage.
- **Block level** — per-block device metrics: I/O patterns, thermal distribution.

6.1.1 VM-Level Physics Metadata

Today's physics model tracks per-word metrics attached to each `DictEntry`:

```
1 typedef struct {
2     uint16_t temperature_q8;    // Thermal state [0x0000, 0xFFFF]
3     uint64_t last_active_ns;    // Last execution timestamp
4     uint32_t entropy;          // Execution randomness
5     int16_t entropy_slope;      // Rate of entropy change
6     uint32_t avg_latency_ns;    // P50 latency
7     /* ... */
8 } DictPhysics;
```

The proposed extension gives each VM its own aggregate physics record. It rolls up thermal state across all words (warmest word, average temperature, count of hot words above 0x8000), execution load (cumulative executions, words-per-second rate, recency), memory pressure (dictionary heap usage against capacity), reliability (error count, error rate, recovery count), child VM health, block-device affinity, and lifecycle state (uptime, age, run state, criticality class).

```
1 typedef struct {
2     uint16_t vm_temperature_q8;    // Warmest word temperature
3     uint16_t vm_avg_temperature_q8; // Average across all words
4     uint32_t vm_hot_word_count;    // Words with temp > 0x8000
```

```

5      uint64_t total_word_executions;
6      uint64_t execution_rate_per_sec;
7      float    dictionary_pressure_pct; // used / max
8      uint32_t error_rate_per_sec;
9      uint32_t active_child_vm_count;
10     uint8_t  vm_state;                // RUNNING, PAUSED, DRAINING,
        DEAD
11     uint8_t  vm_criticality;          // system, critical, normal,
        background
12     /* ... */
13 } VMPhysics;

```

The record attaches directly to the `VM` struct. Every physics event maintains it: `physics_metadata_touch()` updates the word, then resolves the current VM and updates the VM-level aggregates, recomputing running averages and rates.

6.1.2 Block Device Physics Metadata

Block storage (1024 blocks of 1024 bytes) is managed by `block_subsystem.c` (logical-to-physical mapping) and `blkio_factory.c` (pluggable RAM, file, and L4Re backends). Today there is no observability beyond capacity tracking. The proposal attaches a `BlockPhysics` record to each block, tracking thermal state (access frequency), access patterns (read and write counts, P50 and P99 latencies), ownership (owning VM and word), content-type hints, and health (CRC32 checksum, dirty and corruption flags, error count). A device-level aggregate summarizes total reads and writes, hottest and coldest block temperatures, and fragmentation percentage.

Block accessors maintain these metrics inline. On read, the subsystem times the I/O, updates the block’s read count and exponential moving-average latency, recomputes temperature from read frequency, and verifies the CRC32 to detect corruption. On write, it does the symmetric work and refreshes the stored checksum.

6.1.3 Mama’s Responsibilities

Mama Forth runs with elevated privileges along three axes:

- **Observe** — all word executions in children, all block-device I/O, all child lifecycle events, all errors and anomalies.
- **Control** — spawn and kill child VMs, allocate blocks to children, enforce governance policies, redirect critical work, scale up and down with load.
- **Coordinate** — share block resources, prevent deadlock, balance load across children, degrade gracefully under stress.

Mama is itself instrumented with a `MamaPhysics` record covering its own supervision load, child-management counts, block-management activity, emergency events (thermal warnings, memory-pressure events, orphan recoveries), and governance metrics (violations detected, enforcement actions, audit events).

6.1.4 The Supervision Loop

Mama runs a periodic control loop. Each cycle observes every child’s VM-level physics, flags thermal warnings, memory pressure above 90%, and error surges, then inspects every block device for hot blocks and excessive fragmentation. It then makes load-balancing decisions, enforces governance compliance, and sleeps for an adaptive interval derived from its own decision latency.

```

1 void mama_supervision_loop(void) {
2     while (mama_vm.running) {
3         for (int i = 0; i < child_vm_count; i++) {
4             VM *child = child_vms[i];
5             VMPhysics p = child->physics;
6             if (p.vm_temperature_q8 > THERMAL_WARNING)
7                 mama_handle_hot_child(child);
8             if (p.dictionary_pressure_pct > 90)
9                 mama_handle_memory_pressure(child);
10            if (p.error_rate_per_sec > ERROR_THRESHOLD)
11                mama_handle_error_surge(child);
12        }
13        mama_balance_load_across_children();
14        mama_check_governance_compliance();
15        sf_sleep_ns(mama_vm.physics.decision_latency_ns * 2);
16    }
17 }

```

Mama spawns a new child when average child temperature exceeds 0xD000 and capacity remains, allocating a block range to the newcomer and rebalancing the work queue. It kills a child that is unrecoverably faulted, idle, or under unrecoverable memory pressure — first recovering the orphan’s blocks, then tearing the VM down. It rebalances block allocation by grouping each child’s hottest blocks together to reduce fragmentation and keep hot data near its consumer.

6.1.5 Three-Tier Feedback Loop

Metrics flow upward. Word execution updates word temperature and latency; the VM aggregates word metrics into VM state; Mama aggregates VM state into system state, which is logged across all three tiers to the analytics heap. Decisions flow downward. A Mama decision (“Child 2 is too hot, spawn Child 3”) creates a VM and allocates blocks; the affected child adjusts its batch groups and sampling rate; word-level optimization boosts priority for hot words; and the block device responds by relocating cold blocks, compacting fragmentation, and prefetching predicted accesses.

6.1.6 Worked Example: Thermal Emergency

A concrete sequence illustrates the loop. A child VM runs normally until a `BLOCK_WRITE` word heats to 0xC000 and propagates that temperature to the VM. Mama’s next cycle detects the warning together with elevated dictionary pressure and an anomalous error rate of 100 errors per second. Mama responds: it spawns a second child for load shedding, reassigns cold work to it, marks the first child’s hot blocks protected, temporarily raises that child’s heap, and logs the emergency to governance. Within a few seconds the original child cools and stabilizes; the system keeps the new child because it now carries useful work, retiring it only once the first child cools below the safe threshold.

6.1.7 Implementation Phases

- **Phase 1 — Foundation.** Word-level physics and execution hooks are complete; VM-level aggregation and block-device physics remain.
- **Phase 2 — Single-VM Control.** Word-level scheduling and memory hints are complete; VM-level health monitoring and observe-only Mama supervision remain.

- **Phase 3 — Multi-VM Orchestration.** Child spawn and kill, block reallocation, cross-child load balancing, and emergency response.
- **Phase 4 — Mama Optimization.** Mama’s own physics tuning, predictive spawning, guided load balancing, and governance enforcement at scale.

Part III

Physics Runtime Recipes

Chapter 7

Understanding Execution Heat

Chapter 8

Tuning the Rolling Window

8.1 Instrumentation Gaps and Tuning Knobs

This section catalogs the metrics the runtime does *not* yet collect — the blind spots that currently prevent certain tuning knobs from being driven by data. It is a working punch list, organized by subsystem and prioritized for the instrumentation campaign internally nicknamed “twisting the dragon’s tail.”

8.1.1 Dead Code

One routine is defined but never reached. `hotwords_bucket_reorder()` at `physics_hotwords_cache.c:201` implements a bubble sort over `execution_heat` but is never invoked from the lookup path, with no automatic trigger. Whether this is genuine dead code or a forgotten hook remains open. If it were enabled, it would need three metrics: reorder frequency, bubble-sort cost, and a measurement of whether reordering actually improves hit latency.

8.1.2 Stack Metrics (Zero Collection)

No stack instrumentation exists, leaving `STACK_SIZE` tuning blind and tail-call optimization impossible to justify.

Metric	Location	Impact of absence
Data stack peak depth	<code>stack_management.c</code>	cannot size the stack to workload
Return stack peak depth	<code>stack_management.c</code>	cannot measure colon nesting
Underflow attempts	<code>arithmetic_words.c:59-62</code>	fragile paths invisible
Overflow attempts	<code>arithmetic_words.c</code>	no exhaustion early warning

Table 8.1: Missing stack metrics.

8.1.3 Dictionary Metrics (Zero Visibility)

Dictionary behavior is opaque. First-character bucketing across 26 buckets may be unbalanced and would directly affect lookup latency, but no histogram exists to confirm. Dictionary growth velocity, `FORGET` usage, lookup failure rate, and word-name length distribution are likewise untracked, leaving fragmentation, “dictionary full” prediction, error-path frequency, and memory efficiency all unknown. Relevant sites include the dictionary lookup in `vm.c`, `memory_management.c`, and `dictionary_words.c`.

8.1.4 Block I/O Metrics (Completely Dark)

The block subsystem reports nothing. Read and write frequency, writeback frequency (`block_subsystem.c:198-200`), RAM cache hit rate, and dirty block ratio are all DoE phase-1 targets; I/O latency and fragmentation ratio are phase-2 targets. Until these land, writeback behavior — a plausible performance killer — cannot be characterized.

8.1.5 Per-Word Execution Metrics

The inner loop in `vm.c` records no per-word execution frequency, latency distribution, word-category breakdown, or colon-versus-native ratio. These metrics are the validation evidence for the cache and window decisions and the basis for hotspot detection and performance-regression tracking.

8.1.6 Heat Dynamics (Phase 2)

Heat dynamics are largely a phase-2 placeholder in `physics_metadata.c`. A heat decay trace is needed to confirm that decay does not destroy useful pattern information. The heat percentile distribution is stored but never dynamically recalculated, and heat concentration ratio (top-10-word heat over total heat), heat velocity, and heat plateau detection are not tracked at all.

8.1.7 Pipelining Metrics (Instrumented, Unused)

Phase-1 pipelining instrumentation exists but feeds nothing. Prediction accuracy by depth is not measured, context transition frequency is counted but not reported, speculation ROI is not aggregated, misprediction cost is measured but not validated, and pipeline stall reduction is not measured — leaving the binary chop over `TRANSITION_WINDOW_SIZE` and the `SPECULATION_THRESHOLD` tuning without inputs.

8.1.8 Cache Pollution and Cycles

The hot-words cache offers no view into its own churn. Eviction and re-promotion cycles (is the cache thrashing?), false negatives (words evicted before promotion, hinting the threshold is too high), eviction candidate value (was LRU the right policy?), bucket scan depth, and promotion wait time are all uncollected as of day one.

8.1.9 Priorities

The instrumentation work is ranked in four tiers:

- **Critical.** Stack peak depth (minutes to add, large insight); block I/O metrics (writeback behavior is a likely performance killer); dictionary bucket load (imbalance drives the latency tail).
- **High (phase-1 DoE).** Per-word execution frequency; heat decay trace; block I/O hit rate.
- **Medium (phase-2 DoE).** Pipelining accuracy by depth; colon nesting depth; dictionary growth velocity.
- **Nice to have (phase 3).** Heat concentration ratio; word-category breakdown; speculation ROI histogram.

Category	Files	Est. LOC	Est. time
Stack metrics	<code>stack_management.c</code> , <code>vm.h</code>	~30	30 min
Dictionary metrics	<code>vm.c</code> , <code>memory_management.c</code> , <code>vm.h</code>	~50	45 min
Block I/O metrics	<code>block_subsystem.c</code> , <code>vm.h</code>	~80	1 hr
Per-word metrics	<code>vm.c</code> , <code>physics_metadata.c</code> , <code>vm.h</code>	~100	1.5 hr
Heat dynamics	<code>physics_metadata.c</code> , <code>vm.h</code>	~60	1 hr
Pipelining accuracy	<code>physics_pipelining_metrics.c</code> , <code>vm.h</code>	~40	45 min

Table 8.2: Implementation effort estimate per metric category.

Chapter 9

Pipelining and Prefetch

9.1 Context Window and Binary-Chop Tuning

The context window extends transition prediction beyond the immediate predecessor, and a binary-chop search selects the window depth that best predicts the next word. This addresses a single question: how deep into execution history must the predictor look?

9.1.1 The Trade-Off

A one-word window asks “after DUP, what comes next?” and is right roughly 60% of the time, because several successors are plausible. A two-word window asks “after LOOP DUP, what comes next?” and the pattern sharpens toward a single answer near 85%. A four-word window improves further to perhaps 90%, but at twice the per-word memory and a more complex sparse hash, with diminishing returns thereafter.

9.1.2 Knob #6: TRANSITION_WINDOW_SIZE

The window depth is a compile-time knob defaulting to two, chosen as the empirical sweet spot: one word captures only the immediate predecessor, two captures “where we came from” plus “what we are doing,” and four or eight rarely justify the overhead.

```
1 make TRANSITION_WINDOW_SIZE=1    # immediate successor only
2 make TRANSITION_WINDOW_SIZE=2    # 2-word patterns (default)
3 make TRANSITION_WINDOW_SIZE=4    # 4-word patterns
```

The window is a per-word circular buffer of word IDs. As each word executes, its predecessor’s window records the context-to-successor transition, then shifts the new word in and the oldest word out. With a window of two the overhead is about 16 bytes per word; for 200 words the total is roughly 27 KB, negligible beside the transition-heat arrays.

9.1.3 Binary-Chop Search

Phase 2 finds the optimal depth by testing window sizes 1, 2, 4, and 8 and measuring prediction accuracy at each. The search starts at two and doubles while accuracy improves, then reverts to one to establish the baseline, and converges on the best observed size.

```
1 uint32_t suggest_window(current_window, accuracy_at_current) {
2     if (current_window == 2 && accuracy_improves) return 4;
3     if (current_window == 4 && accuracy_improves) return 8;
4     if (current_window >= 4 && accuracy_plateaus) return 1;
5     /* otherwise converge to the size with best accuracy */
6 }
```

For each candidate, the harness rebuilds at that window size, runs the `BENCH-DICT` workload to collect transition data, counts transitions whose context-to-successor probability exceeds 70% as predictable, and records accuracy as the predictable fraction.

9.1.4 Expected Outcome

FORTH exhibits strong local patterns, so most decisions depend on about two words of context. The hypothesis, to be confirmed by measurement:

Window	Predicted accuracy
1	~65%
2	~82% (expected winner)
4	~88% (marginal, 2× memory)
8	~89% (overhead unjustified)

The recommendation is to keep the window at two: 82% accuracy at minimal overhead is the best return on investment.

9.1.5 Interaction With Other Knobs

Window depth shifts the useful settings of the speculation knobs. A deeper, more accurate window produces fewer failures, permitting a lower speculation threshold (more aggressive prefetch) and fewer required samples before a pattern is trusted. A shallow window calls for the opposite: a higher threshold and more samples to compensate for noisier predictions.

9.1.6 Status

The Phase 1 extension—the window knob, the per-word circular buffer, and the context-aware recording functions—is complete and collecting data. Three functions remain stubs awaiting Phase 2: the accuracy-reporting string, the context recording that will build a sparse hash of context-to-successor patterns, and the binary-chop suggestion logic that will react to measured accuracy rather than blindly doubling.

Chapter 10

Reading Heartbeat Output

10.1 Heartbeat CSV Export

The heartbeat subsystem emits real-time multivariate VM metrics to `stderr` in CSV form, exposing StarForth's adaptive runtime to external analysis. Each row carries eleven performance indicators sampled once per heartbeat tick — nominally every millisecond — producing a continuous time series of the running system's internal state.

The design favours streaming over buffering. Metrics are read directly from VM state with no intermediate copy, each row is flushed immediately so that downstream tools see data as it is produced, and the stream is confined to `stderr` to keep `stdout` reserved for test output. The emitting thread is a background `pthread` decoupled from the main interpreter, so capture does not perturb execution timing beyond a fixed, small cost per tick.

10.1.1 CSV Format

The VM emits raw rows only; column names are supplied separately by analysis tooling. The reference header is:

```
1 tick_number,elapsed_ns,tick_interval_ns,cache_hits_delta,
   bucket_hits_delta,
2 word_executions_delta,hot_word_count,avg_word_heat>window_width,
3 predicted_label_hits,estimated_jitter_ns
```

The *Status* column below reflects which fields carry live values and which currently emit a placeholder zero pending delta-tracking work.

10.1.2 Capturing Metrics

The simplest capture redirects `stderr` to a file and discards `stdout`:

```
1 # Capture metrics, discard test output
2 ./build/amd64/fastest/starforth --doe 2>heartbeat.csv 1>/dev/null
3
4 # Keep both streams, separated
5 ./build/amd64/fastest/starforth --doe 1>test_output.txt 2>heartbeat.
   csv
```

Because rows are flushed as they are produced, the stream can be monitored live — for example piped through `csvlook` from `csvkit`, or followed with `tail -f`. For post-processing, prepend the reference header and load the result into R or pandas:

```
1 { echo "tick_number,elapsed_ns,tick_interval_ns,cache_hits_delta,\
2 bucket_hits_delta,word_executions_delta,hot_word_count,avg_word_heat
   ,\
```

Table 10.1: Heartbeat CSV columns

Column	Type	Unit	Description	Status
<code>tick_number</code>	<code>uint32</code>	count	Monotonic tick counter since VM start	live
<code>elapsed_ns</code>	<code>uint64</code>	ns	Time since VM initialization	live
<code>tick_interval_ns</code>	<code>uint64</code>	ns	Time since previous tick (actual vs. nominal 1 ms)	live
<code>cache_hits_delta</code>	<code>uint32</code>	count	Hotwords cache hits since last tick	TODO
<code>bucket_hits_delta</code>	<code>uint32</code>	count	Bucket-level cache hits since last tick	TODO
<code>word_executions_delta</code>	<code>uint32</code>	count	FORTH word executions since last tick	TODO
<code>hot_word_count</code>	<code>uint64</code>	count	Words at or above the heat promotion threshold (≥ 10)	live
<code>avg_word_heat</code>	<code>double</code>	heat	Mean execution heat across the dictionary ($\mathbb{Q}_{48.16} \rightarrow \text{double}$)	live
<code>window_width</code>	<code>uint32</code>	entries	Current rolling window effective size (adaptive)	live
<code>predicted_label_hits</code>	<code>uint32</code>	count	Successful pipelining speculation hits	TODO
<code>estimated_jitter_ns</code>	<code>double</code>	ns	Absolute deviation from the nominal 1 ms interval	live

```

3 window_width,predicted_label_hits,estimated_jitter_ns"; \
4 cat heartbeat.csv; } > analysis.csv

```

10.1.3 Analysis Use Cases

The eleven-dimensional series supports three broad classes of analysis. For dynamics modeling, the trajectory admits phase-space reconstruction to locate attractor basins in the adaptive parameter space, Lyapunov-exponent estimation to quantify stability, and mutual-information analysis to discover coupling between metrics. For tuning validation, window-width adaptation can be correlated against prediction accuracy, heat-decay behaviour against stale word accumulation, and jitter against thread-scheduling effects. For workload profiling, the same data yields hot-word churn rate, execution density (`word_executions_delta` over `tick_interval_ns`), and cache efficiency relative to working-set size.

10.1.4 Implementation

Capture is performed by `heartbeat_capture_tick_snapshot()`, which reads the monotonic tick counter, computes elapsed and inter-tick time, walks the dictionary to count hot words and their mean heat, samples the rolling window size, and estimates jitter as the absolute difference between the actual and nominal interval. The walk is linear in dictionary size (on the order of a few hundred words). Emission is performed by `heartbeat_emit_tick_row()`, which formats the eleven values and flushes `stderr` immediately. Both are invoked once per cycle from the background heartbeat thread.

The thread runs without the original 50 ms startup delay, so metrics begin flowing immediately even for short-running DoE workloads. The trade-off is that the first few ticks may show transient values during the word-registration phase.

10.1.5 Known Limitations

Three groups of fields are not yet wired to live deltas and currently emit zero: the cache, bucket, and word-execution counters all require per-tick baseline tracking in the heartbeat state. Separately, the source notes an uninitialized `run_start_ns`, which can produce implausibly large `elapsed_ns` values in the first ticks until the field is seeded at VM initialization, and a missing aggregation of per-word prefetch hits behind `predicted_label_hits`.

The runtime cost is small: emission adds well under one percent CPU at the nominal tick rate, the snapshot is a stack-allocated structure of under a hundred bytes, and the output volume is on the order of a hundred kilobytes per second of execution.

Chapter 11

Physics Freeze and Thaw

Part IV

Design of Experiments

Chapter 12

The 2^7 Factorial Design

Chapter 13

Running the Factorial Harness

13.1 Quick Start: The 2^6 Factorial DoE

The factorial Design of Experiments runs a complete, exhaustive $2^6 = 64$ -configuration sweep in a single pass, collecting every measurement for later analysis. It toggles six independent binary feedback loops across all combinations and runs each configuration a chosen number of times.

13.1.1 Three Run Sizes

The same script supports three levels of statistical rigor, trading time for precision:

- **Quick validation (30–45 minutes)** — `-runs-per-config 1` gives $64 \times 1 = 64$ runs, enough to confirm the script and infrastructure work.
- **Standard full DoE (2–4 hours)** — `-runs-per-config 30` gives $64 \times 30 = 1,920$ runs, with good statistical power for main effects. This is the production-grade default.
- **High-precision run (6–12 hours)** — `-runs-per-config 100` gives $64 \times 100 = 6,400$ runs, best for detecting subtle interactions; typically run overnight.

```
1 ./scripts/run_factorial_doe.sh --runs-per-config 30 2025
  _11_19_FULL_FACTORIAL
```

13.1.2 What Is Tested

The six factors are independent binary feedback loops, each gated by a build flag.

Loop	Toggle	Function
1	ENABLE_LOOP_1_HEAT_TRACKING	Count execution frequency
2	ENABLE_LOOP_2_ROLLING_WINDOW	Capture execution history
3	ENABLE_LOOP_3_LINEAR_DECAY	Age words over time
4	ENABLE_LOOP_4_PIPELINING_METRICS	Track word transitions
5	ENABLE_LOOP_5_WINDOW_INFERENCE	Infer window width
6	ENABLE_LOOP_6_DECAY_INFERENCE	Infer decay slope

Table 13.1: The six feedback-loop factors in the factorial design.

All 64 combinations are exercised, from the all-off baseline (0_0_0_0_0_0) through single-loop configurations to the all-on configuration (1_1_1_1_1_1, the current optimum).

13.1.3 Generated Artifacts

A completed experiment directory contains the full measurement set (`experiment_results.csv` — for the standard run, 1,920 rows), a timing and metadata summary (`experiment_summary.txt`), the list of all 64 configurations (`configuration_manifest.txt`), the randomized execution order (`test_matrix.txt`), and a per-run log directory.

13.1.4 Execution Behavior

The script builds each of the 64 configurations (rebuilding only when the configuration changes), runs the tests in randomized order to eliminate systematic and thermal bias, collects metrics into CSV, and prints progress after every run. Four design decisions underpin the method: rebuild on every configuration change for clean isolation, randomize execution to remove temporal and thermal bias, run the complete factorial to capture all interactions, and collect everything in one pass for later analysis.

13.1.5 Analysis

After completion the results move to the analysis repository, where the factorial analysis script surfaces the main effects (which individual loops help most), interactions (whether loops help or hinder one another), the optimal subset (the best combination), and the performance-versus-complexity trade-offs.

```
1 python3 analyze_factorial.py data/2025_11_19_FULL_FACTORIAL/
   experiment_results.csv
```

13.2 Complete 2^6 Factorial Design of Experiments

13.2.1 Design Rationale

Incremental tuning — enabling one feedback loop, measuring, then enabling a second — cannot reveal interaction effects. Two loops that each contribute a 3% improvement in isolation may deliver 10% when combined, or may cancel each other. The 2^6 factorial design addresses this by testing *all* 64 binary combinations of the six feedback loops in a single randomised experiment, collecting a unified dataset for post-hoc analysis.

13.2.2 The Six Feedback Loops

Loop	Makefile flag	Purpose
L1	ENABLE_LOOP_1_HEAT_TRACKING	Count word execution frequency
L2	ENABLE_LOOP_2_ROLLING_WINDOW	Capture execution history (circular buffer)
L3	ENABLE_LOOP_3_LINEAR_DECAY	Age words by exponential decay over time
L4	ENABLE_LOOP_4_PIPELINING_METRICS	Track word-to-word transitions
L5	ENABLE_LOOP_5_WINDOW_INFERENCE	Infer optimal window width via Levene's test
L6	ENABLE_LOOP_6_DECAY_INFERENCE	Infer decay slope via linear regression

13.2.3 Configuration Naming

Each configuration is named by a six-digit binary string L1_L2_L3_L4_L5_L6 where each digit is 0 (off) or 1 (on). The baseline 000000 represents pure FORTH-79 with no adaptive physics; 111111 enables all loops.

13.2.4 Execution

Three run levels are available:

```

1 # Minimal validation (~30-45 min, 64 builds x 1 run)
2 ./scripts/run_factorial_doe.sh --runs-per-config 1 TEST_RUN
3
4 # Standard production (~2-4 hours, 64 x 30 = 1,920 runs)
5 ./scripts/run_factorial_doe.sh --runs-per-config 30 FULL_FACTORIAL
6
7 # High-precision overnight (~6-12 hours, 64 x 100 = 6,400 runs)
8 ./scripts/run_factorial_doe.sh --runs-per-config 100
   HIGH_PRECISION_DOE

```

Each configuration requires a clean `make` rebuild to prevent state contamination. All runs are randomised into a single execution matrix to eliminate thermal ramp, temporal, and ordering biases.

13.2.5 Output Structure

The experiment produces a CSV file with one row per run and a set of run logs:

```

1 <experiment_label>/
2   experiment_results.csv           # All measurements (1,920+ rows)
3   experiment_summary.txt          # Timing, metadata, statistics
4   configuration_manifest.txt      # All 64 configs documented
5   test_matrix.txt                # Randomised execution order
6   run_logs/                      # Per-run logs

```

Each CSV row contains a timestamp, configuration identifier, run number, 35+ performance metrics from the test harness, and the six loop-enable flags. This schema allows every metric to be correlated with every loop combination.

13.2.6 Analysis

Main effects. The average change in a response metric when loop k transitions from off to on, marginalised over all other loops:

$$\hat{\beta}_k = \bar{y}_{(L_k=1)} - \bar{y}_{(L_k=0)} \quad (13.1)$$

Two-way interactions. The interaction effect for loops a and b :

$$\hat{\beta}_{ab} = \bar{y}_{(1,1)} - \bar{y}_{(1,0)} - \bar{y}_{(0,1)} + \bar{y}_{(0,0)} \quad (13.2)$$

A positive $\hat{\beta}_{ab}$ indicates synergy; a negative value indicates suppression.

```

1 import pandas as pd
2 df = pd.read_csv("experiment_results.csv")
3
4 factor_cols = ['enable_loop_1_heat_tracking',
5               'enable_loop_2_rolling_window',
6               'enable_loop_3_linear_decay',
7               'enable_loop_4_pipelining_metrics',
8               'enable_loop_5_window_inference',
9               'enable_loop_6_decay_inference']
10 metric = 'vm_workload_duration_ns_q48'
11
12 main_effects = {

```

```

13     col: df[df[col]==1][metric].mean() - df[df[col]==0][metric].mean
14         ()
15     for col in factor_cols
16 }

```

13.2.7 Key Design Principles

- **Rebuild every configuration.** Ensures isolated, uncontaminated measurements for each loop combination.
- **Randomise all runs.** The 1,920 runs are interleaved in a single randomised matrix, not grouped by configuration.
- **Collect once, analyse separately.** Data collection and statistical analysis are strictly separated to prevent confirmation bias.
- **Full factorial, not fractional.** All $2^6 = 64$ combinations are tested; fractional designs would alias higher-order interactions.

13.3 Factorial DoE Analysis Workflow

13.3.1 End-to-End Process

The factorial experiment produces a single CSV file in the StarForth repository that is then analysed in a separate environment:

1. Run `run_factorial_doe.sh` — 64 builds, 1,920+ randomised runs, CSV and logs written to the experiment label directory.
2. Transfer the CSV to the analysis environment.
3. Load into Python or R; separate factor columns from metric columns.
4. Compute main effects and two-way interactions.
5. Identify optimal configuration subsets and generate visualisations.
6. Write the summary report.

13.3.2 Phase 1: Data Collection

The run script handles all 64 configurations:

```

1 # Quick validation (30 min)
2 ./scripts/run_factorial_doe.sh --runs-per-config 1 TEST_01
3
4 # Full production run (2-4 hours)
5 ./scripts/run_factorial_doe.sh --runs-per-config 30 FULL_FACTORIAL

```

Monitor progress while the script runs:

```

1 watch -n 5 'wc -l /path/to/experiment_results.csv'

```

Expected progression for a 30-replicate run: approximately 500 rows after 30 minutes, 1,500 rows after two hours, 1,921 rows at completion.

13.3.3 Phase 2: Data Loading

```

1 import pandas as pd
2 import numpy as np
3
4 df = pd.read_csv("experiment_results.csv")
5 print(f"Loaded {len(df)} runs, "
6       f"{len(df.groupby('configuration'))} configurations")
7
8 factor_cols = ['enable_loop_1_heat_tracking',
9               'enable_loop_2_rolling_window',
10              'enable_loop_3_linear_decay',
11              'enable_loop_4_pipelining_metrics',
12              'enable_loop_5_window_inference',
13              'enable_loop_6_decay_inference']
14
15 metric = 'vm_workload_duration_ns_q48'

```

13.3.4 Phase 3: Main Effects

For each loop, the main effect is the average metric change when the loop transitions from off to on, marginalised over all other loop states:

```

1 main_effects = {}
2 for loop in factor_cols:
3     on_mean = df[df[loop] == 1][metric].mean()
4     off_mean = df[df[loop] == 0][metric].mean()
5     effect = on_mean - off_mean
6     pct = 100.0 * effect / off_mean
7     main_effects[loop] = {'effect': effect, 'pct': pct}
8
9 for loop, e in sorted(main_effects.items(),
10                      key=lambda x: abs(x[1]['pct']), reverse=True):
11     print(f"{loop:40} {e['pct']:+7.2f}%")

```

13.3.5 Phase 4: Two-Way Interactions

The interaction effect for loops a and b :

$$\hat{\beta}_{ab} = \bar{y}_{(1,1)} - \bar{y}_{(1,0)} - \bar{y}_{(0,1)} + \bar{y}_{(0,0)} \quad (13.3)$$

A positive $\hat{\beta}_{ab}$ indicates synergy; negative indicates suppression.

```

1 from itertools import combinations
2
3 for loop_a, loop_b in combinations(factor_cols, 2):
4     y_00 = df[(df[loop_a]==0)&(df[loop_b]==0)][metric].mean()
5     y_01 = df[(df[loop_a]==0)&(df[loop_b]==1)][metric].mean()
6     y_10 = df[(df[loop_a]==1)&(df[loop_b]==0)][metric].mean()
7     y_11 = df[(df[loop_a]==1)&(df[loop_b]==1)][metric].mean()
8     interaction = y_11 - y_10 - y_01 + y_00
9     if abs(interaction) > 100_000:
10         print(f"{loop_a}x{loop_b}: {interaction:+.0f}")

```

13.3.6 Phase 5: Optimal Configuration Search

```
1 # Top 10 configurations by mean performance
2 best = (df.groupby('configuration')[metric]
3         .mean()
4         .nlargest(10)
5         .reset_index())
6 print(best.to_string())
```

13.3.7 Phase 6: Reporting

The analysis report covers four sections:

1. **Main effects table** — individual loop impacts sorted by effect size with p -values.
2. **Interaction matrix** — 6×6 table of two-way interaction estimates; cells highlighted for synergy or suppression.
3. **Optimal configurations** — top 3–5 production candidates on the Pareto frontier of performance versus code complexity.
4. **Statistical summary** — variance by configuration, 95% confidence intervals, regression R^2 .

Chapter 14

Heartbeat DoE

14.1 Quick Start: The Heartbeat DoE

The Heartbeat DoE is the second stage of StarForth’s optimization campaign. Where the factorial sweep ranked configurations by final performance, this experiment measures the *temporal stability* of the five best performers from that baseline. It runs roughly 250 iterations — five elite configurations, fifty runs each — and takes two to four hours plus about thirty minutes of analysis, all from a single command.

14.1.1 Running the Experiment

The launch script builds the five configurations, runs each fifty times while recording heartbeat metrics (jitter, convergence, and coupling), and writes `experiment_results_heartbeat.csv`.

```
1 ./scripts/run_factorial_doe_with_heartbeat.sh 2025
   _11_20_HEARTBEAT_TOP5
```

A short pre-flight check is worthwhile: confirm roughly 1 GB of free disk, verify a clean `make fastest`, and smoke-test the binary with `starforth -c "1 2 + . BYE"` (which should print 3). The script prompts for confirmation before starting and can be interrupted with Ctrl+C — results are saved incrementally, so it resumes.

14.1.2 The Five Configurations

Config	Loops	Character
1	1_0_1_1_1_0	Minimal (heat + decay + pipelining + window inf.)
2	1_0_1_1_1_1	Config 1 plus decay inference
3	1_1_0_1_1_1	Alternative (skips the decay loop)
4	1_0_1_0_1_0	Lean (heat + decay + window only)
5	0_1_1_0_1_1	Contrast (no heat tracking)

Table 14.1: The five elite configurations carried forward from the baseline.

These five were the strongest performers from Stage 1, a 3,200-run baseline. The present stage re-examines them through the lens of stability rather than raw performance.

14.1.3 Analysis

The analysis script loads the CSV, ranks configurations by stability score, generates six visualizations, and identifies the golden configuration.

```
1 Rscript scripts/analyze_heartbeat_stability.R
```

It emits a detailed metrics table (`stability_rankings.csv`) and six PNG plots: overall stability scores, jitter control, convergence speed, load coupling, a metrics heatmap, and a jitter-versus-convergence trade-off.

14.1.4 Key Metrics

Metric	Meaning	Target
Stability score	Overall goodness (0–100)	> 75 (gold), > 70 (ok)
Jitter (CV)	Steadiness of the heartbeat	< 0.15
Convergence	Speed of self-tuning	Within 5000 ticks
Load coupling	Heartbeat tracks workload	> 0.75 correlation

Table 14.2: Heartbeat stability metrics and their targets.

The golden configuration is the one combining the lowest jitter, fastest convergence, and best load coupling.

14.1.5 Troubleshooting

- *Build fails* — inspect the per-config build log; fix and retry, and the experiment resumes.
- *Analysis fails* — install the R dependencies (`tidyverse`, `ggplot2`, `gridExtra`) and retry.
- *All configs look equal* — confirm the heartbeat was compiled in (`nm` on the binary should show heartbeat symbols) and that a run log carries the expected ~60 columns.

14.1.6 Where This Fits

Stage 1 ran the 2⁶ factorial with fifty repetitions and selected the five best configurations. Stage 2, this experiment, adds heartbeat observability to those five and names the golden configuration by stability. Stage 3 adopts that golden configuration as the baseline for the MamaForth experiments and further optimization rounds.

14.2 Heartbeat DoE Execution Guide

14.2.1 Overview

The heartbeat DoE executes five elite configurations at 50 replicates each (250 total runs) with the heartbeat thread enabled. The experiment requires a clean build environment, approximately 600 MB of storage for run logs and CSV output, and 2–4 hours of uninterrupted machine time.

14.2.2 Pre-Execution Checklist

1. **Build system.** Verify the build completes without errors:

```
1 make clean
2 make fastest HEARTBEAT_THREAD_ENABLED=1
3 ./build/amd64/fastest/starforth -c "1_2_+_._BYE"
```

2. **Storage.** Confirm at least 1 GB is available in the experiment output directory. The CSV file reaches approximately 50 MB; run logs reach approximately 500 MB.
3. **System stability.** The experiment runs continuously for 2–4 hours. Background CPU load from compilation jobs, browser processes, or automatic system updates biases heartbeat measurements. Minimise competing load before launching.
4. **Heartbeat symbols.** Confirm the heartbeat thread is compiled in:

```
1 nm ./build/amd64/fastest/starforth | grep heartbeat
```

14.2.3 Execution

```
1 # Standard run (50 replicates per configuration, ~2-4 hours)
2 ./scripts/run_factorial_doe_with_heartbeat.sh HEARTBEAT_TOP5
3
4 # Extended run (100 replicates, ~6-8 hours)
5 ./scripts/run_factorial_doe_with_heartbeat.sh --runs-per-config 100
   HEARTBEAT_EXTENDED
```

The script prints a configuration manifest and a preview of the randomised test matrix before execution begins, then prompts for confirmation. Each run takes approximately 10–15 seconds.

14.2.4 Output Verification

On completion, verify the output file:

```
1 wc -l experiment_results_heartbeat.csv
2 # Expected: 251 (header + 250 data rows)
3
4 head -1 experiment_results_heartbeat.csv
5 # Should include: total_heartbeat_ticks, tick_interval_mean_ns,
6 #   tick_interval_cv, load_interval_correlation,
   overall_stability_score
```

14.2.5 Analysis

```
1 Rscript scripts/analyze_heartbeat_stability.R \
2   /path/to/experiment_results_heartbeat.csv
```

The analysis script produces six visualisations and a `stability_rankings.csv` summary. The golden configuration is the one with the highest composite stability score, subject to $\rho_{\text{load-heartrate}} \geq 0.75$.

14.2.6 Metric Interpretation

Metric	Target	Interpretation
Stability score	$\geq 75/100$	Production-ready configuration
Jitter CV	< 0.15	Heartbeat variation within 15% of mean
Load correlation	> 0.75	Heartrate responds strongly to workload
Convergence	$< 5,000$ ticks	Adapts within the test window

A configuration scoring below 65 on any primary axis was likely already excluded by Stage 1 analysis. The golden configuration emerges from clear statistical separation at the top of the stability ranking.

14.2.7 Troubleshooting

All configs score identically. The heartbeat thread may have been compiled out. Verify `HEARTBEAT_THREAD_ENABLED=1` at build time and confirm non-zero values in the `tick_interval_*` columns.

Runtime crashes for specific configurations. Examine the per-run log for the failing configuration. Common causes are thread-safety issues in heartbeat metrics collection or unguarded rolling-window access. See Section ?? for the incompatibility handling framework.

R package errors. Install required packages before running the analysis script:

```
1 install.packages(c("tidyverse", "ggplot2", "gridExtra"))
```

Chapter 15

Optimization DoE

15.1 Quick Start: The Optimization DoE

The Optimization DoE refines individual runtime parameters one at a time. It runs five focused experiments — each called an “opportunity” — that test a handful of parameter variations and report which value performs best. The full set completes in roughly nine minutes.

15.1.1 The Five Opportunities

Each opportunity is launched with `--opportunity N` and a label; piping an empty line past the confirmation prompt lets them run unattended.

```
1 echo "" | ./scripts/run_optimization_doe.sh --opportunity 1
  OPP_01_DECAY_SLOPE
2 echo "" | ./scripts/run_optimization_doe.sh --opportunity 2
  OPP_02_WINDOW_WIDTH
3 echo "" | ./scripts/run_optimization_doe.sh --opportunity 3
  OPP_03_DECAY_RATE
4 echo "" | ./scripts/run_optimization_doe.sh --opportunity 4
  OPP_04_WINDOW_SIZING
5 echo "" | ./scripts/run_optimization_doe.sh --opportunity 5
  OPP_05_THRESHOLD
```

Each script tests three or four parameter variations, runs two iterations of thirty samples per configuration (sixty samples in total), emits metrics in $\mathbb{Q}_{48.16}$ fixed-point integer math, and writes a CSV of results.

15.1.2 Picking a Winner

For each opportunity, the winning configuration is the one with the highest `cache_hit_percent` and the lowest `vm_workload_duration_ns_q48`. Results can be inspected directly by sorting the CSV on the workload-duration column (lower is better) or analyzed in R with the `load-and-explore` script.

```
1 tail -n +2 .../OPP_01_DECAY_SLOPE/experiment_results.csv | \
2 sort -t',' -k27 -n | head -5
```

15.1.3 Reading $\mathbb{Q}_{48.16}$ Metrics

All metrics are $\mathbb{Q}_{48.16}$ fixed-point integers: a 48-bit integer part and a 16-bit fraction. To recover a decimal value, divide by 65536. For example, a `vm_workload_duration_ns_q48` of 315,797,667,840 corresponds to $315,797,667,840/65536 = 4,822,021$ nanoseconds, and a `cpu_freq_delta_mhz` of $-6,356,992$ corresponds to $-6,356,992/65536 = -97.07$ MHz.

15.1.4 Recommended Sequencing

The opportunities are best run in order, reviewing results between each so that a chosen value can be locked before moving on. Opportunities 1 through 3 each pick and lock a single winner; opportunity 4 analyzes interactions; and opportunity 5 applies the final refinement. The findings then aggregate into an optimized baseline.

Opportunity	Expected impact	Expected winner
#1 Decay slope	8–15%	DECAY_SLOPE_0.33 or 0.5
#2 Window width	6–12%	WINDOW_SIZE_4096 or 8192
#3 Decay rate	3–6%	Likely NORMAL (baseline)
#4 Window sizing	5–8%	Likely WINDOW_8K_DECAY_0.33
#5 Threshold	2–4%	Likely THRESHOLD_10 (baseline)

Table 15.1: Expected impact and likely winner per opportunity.

15.1.5 Troubleshooting

- *Hangs at the confirmation prompt* — press Enter, or pipe an empty line in with `echo "" |`.
- *Binary not found* — rebuild with `make test`.
- *No CSV data* — inspect the per-run logs in the experiment’s `run_logs/` directory.

15.2 Physics Optimization DoE Guide

15.2.1 Overview

Five optimisation opportunities in the StarForth physics engine are tested sequentially using progressive design-of-experiments: each opportunity explores 3–4 parameter variations at 60 samples per configuration (two iterations of 30 samples), and the winner is locked in before proceeding to the next opportunity.

15.2.2 Q48.16 Fixed-Point Format

All metrics are reported in Q48.16 fixed-point representation: 48 integer bits and 16 fractional bits. This format is deterministic, free of floating-point rounding, and formally verifiable.

Decimal value	Q48.16 integer	Conversion
0.2	13107	$0.2 \times 65536 = 13107.2$
0.33	21627	$0.33 \times 65536 = 21626.9$
0.5	32768	$0.5 \times 65536 = 32768$
0.7	45875	$0.7 \times 65536 = 45875.2$

To convert a Q48.16 CSV value to a decimal: divide by 65536. For example, `vm_workload_duration_ns_q48 = 315,797,667,840` corresponds to $315,797,667,840/65536 \approx 4,822,021$ nanoseconds.

15.2.3 Optimisation Opportunities

Opportunity 1 — Decay Slope Inference. Four decay slope values in Q48.16 are tested: 13107 (0.2), 21627 (0.33, baseline), 32768 (0.5), 45875 (0.7). The decay slope controls how rapidly execution heat dissipates from the hot-words cache. Expected improvement: 8–15%. Decision criterion: highest cache hit rate combined with lowest workload duration.

Opportunity 2 — Variance-Based Window Width Tuning. Three rolling window sizes are tested: 2048, 4096 (baseline), 8192 bytes. Window size determines how many execution events are retained for adaptive decisions. Expected improvement: 6–12%. Decision criterion: best balance of context prediction accuracy and workload duration.

Opportunity 3 — Decay Rate Parameter Tuning. Three combinations of decay interval (nanoseconds) and adaptive shrink rate are tested: fast/fast (500 ns, shrink 50), normal/normal (1000 ns, shrink 75, baseline), slow/slow (2000 ns, shrink 100). Expected improvement: 3–6%.

Opportunity 4 — Window × Decay Interaction. A 2×2 factorial crossing two window sizes (2048, 8192) with two decay slopes (0.33, 0.5) reveals interaction effects between the two parameters. Expected improvement: 5–8%.

Opportunity 5 — Hot-Words Cache Threshold. Four cache-promotion thresholds are tested: 5, 10 (baseline), 20, 50 execution-count units. Lower thresholds promote more words to the cache; higher thresholds produce a leaner, higher-precision cache. Expected improvement: 2–4%.

15.2.4 Execution

```

1  # Opportunity 1: Decay Slope (4 configs x 60 runs = 240 total)
2  ./scripts/run_optimization_doe.sh --opportunity 1 OPP_01_DECAY_SLOPE
3
4  # Opportunity 2: Window Width (3 configs x 60 runs = 180 total)
5  ./scripts/run_optimization_doe.sh --opportunity 2
6    OPP_02_WINDOW_WIDTH
7  # ... opportunities 3-5 follow the same pattern

```

All five opportunities complete in approximately 9 minutes at two iterations. To increase statistical power, add `-exp-iterations 4` (doubles samples per configuration).

15.2.5 Analysis

For each opportunity, sort the CSV by `vm_workload_duration_ns_q48` ascending to identify the fastest configuration:

```

1  tail -n +2 experiment_results.csv | sort -t',' -k27 -n | head -5

```

The winner is then locked into the Makefile before proceeding to the next opportunity. After all five opportunities are completed, a validation run confirms the cumulative improvement over the pre-optimisation baseline.

15.2.6 Expected Cumulative Improvement

Sequential optimisation of all five opportunities is expected to yield a 15–30% cumulative reduction in `vm_workload_duration_ns_q48` compared to the untuned default configuration.

15.3 Physics Engine Validation — Execution Guide

15.3.1 Overview

The comprehensive physics engine validation experiment runs 90 total measurements across three configurations at 30 replicates each. The benchmark executes 100,000 dictionary lookups per run; estimated wall-clock time is 2–3 hours including three separate clean builds.

Configuration	Flags	Purpose
A_BASELINE	ENABLE_HOTWORDS_CACHE=0 ENABLE_PIPELINING=0	Control
B_CACHE	ENABLE_HOTWORDS_CACHE=1 ENABLE_PIPELINING=0	Cache only
C_FULL	ENABLE_HOTWORDS_CACHE=1 ENABLE_PIPELINING=1	Full physics

15.3.2 Execution

```

1 # Run with default output directory
2 ./scripts/run_comprehensive_physics_experiment.sh
3
4 # Run with explicit output path
5 ./scripts/run_comprehensive_physics_experiment.sh ./physics_results
6
7 # Analyse results when complete
8 python3 scripts/analyze_physics_experiment.py \
9     ./physics_results/experiment_results.csv \
10     --output analysis_report.md

```

Monitor progress while running:

```

1 watch -n 5 'wc -l physics_results/experiment_results.csv'
2 # Expected: 91 rows when complete (1 header + 90 data)

```

15.3.3 Output Schema

The CSV contains one row per run. Key columns:

Column	Type	Notes
configuration	string	A_BASELINE, B_CACHE, C_FULL
cache_hit_percent	float	Hit rate (0–100)
context_accuracy_percent	float	Pipelining prediction accuracy
total_runtime_ms	float	Wall-clock run time
speedup_vs_baseline	float	Computed in analysis script
ci_lower_95	float	Lower 95% credible interval
ci_upper_95	float	Upper 95% credible interval

15.3.4 Success Criteria

- **A_BASELINE**: All 30 runs complete; consistent execution times; coefficient of variation below 10%.
- **B_CACHE**: Cache hit rate above 20%; speedup 95% credible interval excludes 1.0; CV below 10%.
- **C_FULL**: Prediction accuracy above 60%; speedup exceeds B_CACHE; pattern diversity saturation above 90%.

15.3.5 System Stability

For reproducible measurements: stop background compilation jobs and file indexers before launching; use the **fastest** build profile (default in the script); run in a dedicated terminal with minimal competing I/O.

15.3.6 Publication Notes

Results should be reported as:

- Point estimate (mean speedup).
- 95% credible interval.
- Sample size ($n = 30$ per configuration).
- Hardware description (CPU, RAM, OS, build profile).

Raw CSV and per-run logs should be archived alongside the analysis report for reproducibility.

Part V

Security and ACL Recipes

Chapter 16

Word-Level ACL Basics

16.1 Word-Level ACL System

StarForth’s word-level access control system combines a thin C infrastructure layer with a pure-FORTH policy capsule, `ACL.4th`. The C layer supplies four `DictEntry` fields for the hot path — a TTL counter, an allow bit, a mode, and a pin — while all policy, all adaptive logic, and all inheritance rules live as colon definitions in `ACL.4th`. A bootstrap superuser, `zuse.4th`, provides the sole authenticated escalation path. The system is implemented through Phase 6; Phase 7 (LithosAnanke parity) remains. The implementation spans `capsules/ACL.4th`, `capsules/zuse.4th`, `src/word_source/acl_words.c`, and `src/test_runner/modules/acl_words_test.c`.

16.1.1 Three Enforcement Dispositions

Each dictionary word holds one of three dispositions:

State	Behavior
STRICT	ACL re-checked on every execution
TTL	ACL cached; re-checked only when the TTL counter reaches zero
PINNED	Mode and decision frozen permanently; a one-way ratchet

16.1.2 Statistically Adaptive TTL

The TTL is not a fixed value. It is derived and continuously updated from the existing SSM execution physics. Hotter words (higher `execution_heat`) earn longer TTLs, amortizing check cost over more executions. A sudden shift in caller pattern or access rate, detected through the rolling window, shrinks the TTL aggressively. Decay pulls a quiescent word’s TTL back down so the next burst re-validates early. The inference engine (Loops L5/L6) detects an oscillating TTL and stabilizes it, using the same coefficient-of-variation threshold logic already employed for window-width inference. PINNED is the asymptote: once the adaptive process converges, the operator thumbtacks it and the accumulator freezes permanently. A security word selects the mode per word:

```
1 ' MY-WORD ACL-STRICT      \ check every execution
2 ' MY-WORD ACL-TTL-MODE    \ use adaptive TTL
3 ' MY-WORD ACL-PIN         \ freeze -- mode and decision immutable
   forever
```

16.1.3 The Pin Ratchet

ACL-PIN (`xt` -) is a one-way ratchet. It transitions STRICT or TTL to PINNED and never back. Once pinned, a word’s `acl_mode` and `acl_ttl` are immutable within that VM context, and any

attempt to alter a pinned word's ACL is silently ignored or errors, depending on policy. Kernel primitive words — BIRTH, EXEC, BYE, and the like — are pinned by Mama at boot before the first BIRTH fires.

16.1.4 Inheritance at Birth

When a child VM is born, ACL state propagates downward once:

1	<code>acl_mode</code>	<code>= parent->acl_mode</code>	<code>\ policy propagates (STRICT or TTL)</code>
2	<code>acl_pinned</code>	<code>= 0</code>	<code>\ always clear -- child owns its own lock</code>
3	<code>acl_ttl</code>	<code>= default</code>	<code>\ reset; child has no execution history</code>
4	<code>acl_decision</code>	<code>= ALLOW</code>	<code>\ re-evaluated on first access</code>

Two properties govern the design. The pin is contextual, not viral: a parent's pinned words do not force the child to be pinned — the child inherits the mode as a starting point and may tighten, relax, or re-pin freely. The security lattice flows downward at birth only; afterward each VM's ACL state is independent, so a compromised child cannot bootstrap its way back to Mama's pinned ACLs.

16.1.5 Interpreter Hook: Two-Level Check

The C interpreter reads only two fields per DictEntry:

1	<code>if (vm->emergency_console) goto execute;</code>	<code>/* 100% bypass -- physical access */</code>
2	<code>if (entry->acl_ttl-- > 0) goto execute;</code>	<code>/* TTL good -- single decrement */</code>
3	<code>acl_recheck(vm, entry);</code>	<code>/* TTL=0: call FORTH ACL-RECHECK */</code>
4	<code>if (!entry->acl_allow) goto reject;</code>	

The hot path costs one decrement and a branch — essentially free. The cold path calls ACL-RECHECK in ACL.4th, which recomputes the adaptive TTL, updates `acl_allow`, and resets the counter. The C side never reasons about policy; it only reads the result.

16.1.6 Two-Console Architecture

Two permanent, independent console layers exist at all times.

The **emergency console** is always present. Its `emergency_console` flag (a `uint8_t` in the VM struct) is written only by C — no FORTH word sets it — and is checked first in the interpreter hot path, producing a 100% ACL bypass; `acl_recheck()` saves and restores it for re-entrancy protection. The `EMERGENCY_CONSOLE_ENABLED` build flag (default 1) strips the interactive fallthrough when set to 0, calling the weak `vm_fault_handler()` symbol instead (overridable for hardware reset, watchdog, or debug probe). BYE returns to the emergency console; only `panic` kills the VM; the prompt is `ok>`.

The **Zuse console** is omnipresent and awaits authentication. Its `zuse_session` flag (also a C-only `uint8_t`) grants a full ACL bypass when a Zuse session is active. The prompt is `zuse)ok>`. The two consoles are completely independent — setting one does not affect the other.

16.1.7 Superuser: Zuse

Named for Konrad Zuse, pioneer of programmable computers, Zuse is the bootstrap superuser — the sole entity that can own the Zuse console and mint user credentials. It is defined in

`capsules/zuse.4th` and loaded by `ACL.4th` at capsule boot. For now it is software-only (no thumbdrive), yet a capsule citizen from day one. Zuse's words are pinned by `ACL-ZUSE-BOOT` at load time. `ACL-BOOT` runs first, then `S" zuse.4th" EXEC`, both from within `ACL.4th` itself (Block 2067). In future, thumbdrive PKI (Ed25519 challenge-response) will replace the software path, with `zuse.4th` becoming the bootstrapper that validates the physical drive.

16.1.8 CA Root

The certificate-authority root is embedded in `ACL.4th` (Block 2066) as the constants `ACL-CA-KEY-LO` and `ACL-CA-KEY-HI`. Because the capsule hash serves as the root-of-trust fingerprint, any change to the CA key changes the capsule hash, and the birth protocol detects the tampering. The key is a zero placeholder until `tools/mkcapsule` embeds the real Ed25519 key at build time.

16.1.9 Security Model and No-Security Condition

Security is opt-in through `init.4th`, gated by a single commented-out line:

```
1 \ S" ACL.4th" EXEC
```

ACL.4th	zuse.4th	Result
absent	absent	No security — open dev mode
present	absent	ACL enforced; emergency REPL locked until Zuse provisioned
present	present	Full lockdown; Zuse owns the Zuse console

Uncommenting the line enables full lockdown; leaving it commented keeps a development build open.

16.1.10 Self-Activating Capsule

`ACL.4th` is self-activating — `init.4th` needs only `S" ACL.4th" EXEC`. Internally, Block 2066 holds the CA root placeholder constants and Block 2067 calls `ACL-BOOT` and then `S" zuse.4th" EXEC`. `ACL-BOOT` stamps default ACL entries onto every existing dictionary word; after it runs, each subsequent `:` definition receives an ACL entry through the defining-word hook, so no word can exist without one.

16.1.11 ACL.4th: Pure-FORTH Implementation

The ACL table is a `CREATED` FORTH array indexed by execution token (XT); `'` (tick) yields the XT of any word, and the XT is simply a cell value usable as a table key.

Word	Stack	Description
<code>ACL-ENTRY</code>	<code>(xt - addr)</code>	O(1) table lookup by XT
<code>ACL-MODE@</code>	<code>(xt - mode)</code>	Read enforcement mode
<code>ACL-MODE!</code>	<code>(mode xt -)</code>	Set mode (no-op if pinned)
<code>ACL-PINNED?</code>	<code>(xt - flag)</code>	Test pin bit
<code>ACL-PIN</code>	<code>(xt -)</code>	Set pin — one-way, irreversible
<code>ACL-STRICT</code>	<code>(xt -)</code>	Set STRICT mode (no-op if pinned)
<code>ACL-TTL-MODE</code>	<code>(xt -)</code>	Set TTL mode (no-op if pinned)
<code>ACL-TTL@</code>	<code>(xt - n)</code>	Read current TTL counter
<code>ACL-TTL!</code>	<code>(n xt -)</code>	Write TTL counter
<code>ACL-ALLOW@</code>	<code>(xt - flag)</code>	Read cached decision
<code>ACL-ALLOW!</code>	<code>(flag xt -)</code>	Write decision
<code>ACL-INHERIT</code>	<code>(src dst -)</code>	Copy mode, clear pin, reset ttl+decision
<code>ACL-RECHECK</code>	<code>(xt -)</code>	Adaptive TTL recompute; update allow + new TTL
<code>ACL-INIT-PRIMITIVES</code>	<code>(-)</code>	Bulk-initialize entries for all existing words

A typical boot-time pin in `init.4th`:

```

1  ' BIRTH  ACL-STRICT  ' BIRTH  ACL-PIN
2  ' EXEC   ACL-STRICT  ' EXEC   ACL-PIN
3  ' BYE    ACL-STRICT  ' BYE    ACL-PIN

```

16.1.12 Bootstrap Sequence

```

1  \ In init.4th (one line, opt-in toggle -- uncomment to enable):
2  S"_ACL.4th" EXEC          \ self-activating: ACL-BOOT then loads zuse.4
                             th
3
4  \ ACL.4th Block 2067 does internally:
5  ACL-BOOT                 \ stamp default ACLs on all existing words
6  S"_zuse.4th" EXEC        \ load and pin Zuse's words
7
8  \ Subsequent capsule loads get ACL entries via the : hook
9  S"_doe.4th" EXEC

```

Every `:` definition after `ACL-BOOT` creates its own ACL entry at definition time via the defining-word hook, so no word is ever born without one.

16.1.13 Capsule Namespace

File	Role
<code>init.4th</code>	Mama VM personality (default); contains the opt-in toggle
<code>init-*.4th</code>	Alternate personalities
<code>doe.4th</code>	DoE experiment harness
<code>ACL.4th</code>	Word-level ACL subsystem; self-activating; loads <code>zuse.4th</code>
<code>zuse.4th</code>	Bootstrap superuser; words pinned by <code>ACL-ZUSE-BOOT</code>
<code>std-blob.4th</code>	Standard library layer (future)

16.1.14 Implementation Status

Phases 1 through 6 are complete on `master`. Phase 1 added the four `DictEntry` fields (`acl_ttl`, `acl_allow`, `acl_mode`, `acl_pinned`), the `emergency_console` and `zuse_session` VM flags, the two-level interpreter check, and `acl_recheck()` with re-entrancy protection. Phase 2 delivered the twelve C primitive words, the XT-indexed table, the field accessors, the pin ratchet, the mode setters, `ACL-INHERIT`, `ACL-RECHECK`, `ACL-INIT-PRIMITIVES`, and the pinning of privileged words. Phase 3 made `ACL.4th` self-activating with the CA placeholder and Zuse load. Phase 4 added the `init.4th` opt-in toggle. Phase 5 contributed POST tests in `acl_words_test.c` (800/800 passing) covering pin monotonicity, inheritance, emergency bypass, `STRICT` and `TTL` behavior, the adaptive accumulator, and persistence of boot-time pins. Phase 6 added five Isabelle/HOL proofs:

- `ACL_Pin_Monotone.thy` — the pin bit is set-only; no operation clears it once set.
- `ACL_Inherit_Clears_Pin.thy` — `ACL-INHERIT` always produces an entry with `acl_pinned = 0`, regardless of source state.
- `ACL_TTL_Bounded.thy` — the `TTL` counter is bounded above by `ACL-TTL-COMPUTE` output and cannot grow unboundedly.
- `ACL_Emergency_Bypass.thy` — when `emergency_console = 1`, the allow/deny decision is never consulted.

- `ACL_No_Escalation.thy` — a child VM cannot produce a pinned entry with higher privilege than its inherited mode.

16.1.15 Remaining Work

Phase 7 (LithosAnanke parity) ports the subsystem to the `lithosananke` branch: verifying that `ACL.4th` loads cleanly in the freestanding kernel context, running `ACL-BOOT` at kernel boot before the first `BIRTH`, wiring `vm->emergency_console` and `vm->zuse_session` to the kernel REPL and Zuse authentication paths, and achieving three-architecture acceptance (`amd64`, `aarch64`, `riscv64` booting to `ok>` with `ACL` active and no regressions), with acceptance logs committed. Phase 8 (future) introduces thumbdrive PKI: Ed25519 challenge-response for the Zuse drive, build-time embedding of the real CA key into Block 2066, CA-signed user minting with a home-block image, and a deliberate no-software-recovery policy — lose the drive and the administrator mints a new one.

Chapter 17

The Zuse Superuser Console

Chapter 18

Emergency Console Fallback

Appendix A

Build Flags Quick Reference

A.1 Build Options Reference

StarForth exposes a comprehensive set of build-time configuration options controlling optimizations, feature selection, platform targeting, and debugging. All options are passed as preprocessor defines via `CFLAGS` or as Makefile variables. Architecture detection runs automatically at compile time through `arch_detect.h`.

A.1.1 Performance Optimization Options

`USE_ASM_OPT`

Enables hand-optimized, architecture-specific assembly implementations for performance-critical operations. Type: integer (0 or 1). Default: 0.

Affected routines include:

- `vm_push` / `vm_pop` — data stack operations
- `vm_rpush` / `vm_rpop` — return stack operations
- `vm_add_check_overflow` / `vm_sub_check_overflow` — arithmetic with overflow detection
- `vm_mul` / `vm_div` — multiply and divide
- `vm_strcmp_asm` / `vm_memcpy_asm` — string and memory operations
- `vm_min_asm` / `vm_max_asm` — conditional-move optimizations
- `vm_abs_asm` — absolute value (ARM64)
- `vm_clz` / `vm_ctz` — count leading/trailing zeros (ARM64)
- `vm_prefetch` / `vm_prefetch_stream` — cache prefetching (ARM64)

Typical performance impact: $\approx 20\text{--}25\%$ on `x86_64`; $\approx 15\text{--}20\%$ on ARM64. Enabled automatically by the `fastest`, `fast`, `turbo`, and `pgo` targets.

`USE_DIRECT_THREADING`

Switches the inner interpreter from indirect threading (the FORTH-79 default) to direct threading using GCC computed-goto (`&&label` / `goto *ptr`). In direct-threaded mode each word's code field points directly to native code, eliminating one indirection level per dispatch. Type: integer (0 or 1). Default: 0.

Requirements: GCC or Clang; must be combined with `USE_ASM_OPT=1`. Full support is provided for x86_64 (`vm_inner_interp_asm.h`) and ARM64 (`vm_inner_interp_arm64.h`).

Typical performance impact: $\approx 30\text{--}40\%$ over indirect threading when combined with `USE_ASM_OPT=1`. Enabled by `fastest`, `fast`, and `pgo` targets.

NDEBUG

Standard C99 macro. When defined, all `assert()` calls compile to nothing, and StarForth additionally suppresses debug logging in hot paths. Default: undefined (assertions active). Typical impact: $\approx 5\text{--}10\%$ speedup.

STARFORTH_PERFORMANCE

Experimental flag that trades safety for throughput. Reduces stack-bounds checking in hot paths (DUP, SWAP, etc.) and applies `UNLIKELY()` branch hints, assuming well-formed Forth input.

Caution: stack overflows may go undetected. Use only on code that has been exhaustively tested. Typical additional gain: $\approx 5\text{--}8\%$ over standard `-O3`. Relevant sources: `src/word_source/stack_words.c:5` (DUP), `src/word_source/stack_words.c:117` (SWAP).

A.1.2 Architecture Options

Architecture detection is automatic via `arch_detect.h`; manual overrides are available for cross-compilation scenarios.

ARCH_X86_64

Targets x86-64 (AMD64 / Intel 64). Auto-detected from compiler predefined macros (`__x86_64__`, `_M_X64`, `__amd64__`). Enables x86_64 assembly optimizations, SSE4.2 string instructions, and the x86_64 direct-threading header.

ARCH_ARM64

Targets AArch64 (ARMv8-A). Auto-detected from `__aarch64__`, `_M_ARM64`, and `__arm64__`. Enables NEON SIMD instructions, ARM64-specific instructions (CLZ, CTZ, RBIT, REV), ARM64 assembly optimizations, and ARM64 direct threading.

Native ARM64 build: `make rpi4`. Cross-compile from x86_64: `make rpi4-cross`.

ARCH_NAME

Human-readable string set automatically: `"x86_64"`, `"ARM64"`, or `"Unknown"`. Displayed by `make help`.

On an unrecognized architecture the build emits a compiler warning and falls back to pure-C implementations; no assembly optimizations are loaded.

A.1.3 Platform / Target Options

STARFORTH_MINIMAL

Enables minimal/freestanding build suitable for bare-metal or microkernel environments. When defined: ANSI color codes are suppressed in logging; I/O is reduced to minimal output; the build links with `-nostdlib -ffreestanding`. Enabled by `make minimal` and `make 14re`.

L4RE_TARGET

Targets the L4Re microkernel operating system. Enables the L4Re block-storage backend, ROMFS file access in place of POSIX, and implies `STARFORTH_MINIMAL=1`. Status: partially implemented (ROMFS stub present). See `src/word_source/starforth_words.c:225` and `include/blkio_fa`

A.1.4 Debugging and Profiling Options**DEBUG**

Debug builds compile with `-O0 -g -DDEBUG`: no optimization, full debugging symbols, all assertions enabled, verbose logging throughout. Enabled by `make debug`.

PROFILE_ENABLED

Activates StarForth's built-in profiler. Type: integer (0 or 1). Default: 0. The profiler tracks per-word execution counts, call-graph relationships (caller/callee), stack operation counts, and total instruction counts. Overhead: $\approx 10\text{--}15\%$.

```
1 ./build/starforth --profile 2          # enable at depth 2
2 ./build/starforth --profile-report # print results
```

See `include/profiler.h`.

STRICT_PTR

Controls the Forth address-to-C pointer mapping. Type: integer (0 or 1). Default: 1.

- `STRICT_PTR=1` — Forth addresses are real pointers (`cell_t = uintptr_t`).
- `STRICT_PTR=0` — Forth addresses are indices/offsets (segmented memory models).

Disable only for benchmarking or non-hosted targets. See `src/vm_api.c:202` and `Makefile:37`.

A.1.5 Block System Options

- `BLKCFG_DEFAULT_FBS` — default Forth block size in bytes (default: 1024). See `include/blkcfg.h`.
- `BLKCFG_PATH_MAX` — maximum path length for block-storage files (default: 4096).
- `BLK_FORTH_SYS_RESERVED` — number of system-reserved RAM blocks, hidden from user access (default: 32, blocks 0–31).
- `BLK_DISK_SYS_RESERVED` — number of system-reserved disk blocks (default: 32, PBN 1024–1055).

A.1.6 Memory Configuration Options

- `DATA_STACK_SIZE` — data stack depth in cells (default: 256). See `include/vm.h`.
- `RETURN_STACK_SIZE` — return stack depth in cells (default: 256).
- `STARFORTH_STATE_BYTES` — total state-buffer size for minimal builds (default: 8192). See `src/main.c:40–42`.
- `SF_FC_BUCKETS` — number of forget-chain hash buckets; defined in `vm.h`.

A.1.7 Feature Flags

STARFORTH_ANSI

Enables ANSI escape sequences for terminal control in the block editor (LIST, EDIT): screen clear (`\x1b[2J`), cursor positioning, and colored line numbers. See `src/word_source/editor_words.c:74,188,20`

VM_HAS_CURRENT_ENTRY

Enables the `vm->current_entry` field, which tracks the currently-compiling word. Required for the SEE decompiler, recursive word definitions, and dictionary introspection. Default: defined (enabled).

A.1.8 Build System Variables

- **CC** — C compiler. Default: `gcc`. Also accepts `clang` and `aarch64-linux-gnu-gcc`.
- **CFLAGS** — compiler flags. Extends Makefile `BASE_CFLAGS`.
- **LDFLAGS** — linker flags. Default includes `-Wl,-gc-sections -s -flto=auto -fuse-linker-plugin -static`.
- **MINIMAL** — integer (0 or 1). Setting `MINIMAL=1` enables `STARFORTH_MINIMAL` and adds `-nostdlib -ffreestanding`.
- **ASM** — integer (0 or 1). When set to 1, the build emits `.s` assembly listings alongside `.o` object files.

A.1.9 Build Configuration Matrix

Table A.1: Common build configurations

Target	USE_ASM_OPT	USE_DIRECT_THREADING	NDEBUG	Opt level	Use case
<code>debug</code>	0	0	No	<code>-O0 -g</code>	Development
<code>all</code>	1	0	No	<code>-O2</code>	Default build
<code>turbo</code>	1	0	Yes	<code>-O3 -flto</code>	Fast, no debug
<code>fast</code>	1	1	Yes	<code>-O3</code>	Fast, readable
<code>fastest</code>	1	1	Yes	<code>-O3 -flto</code>	Maximum speed
<code>pgo</code>	1	1	Yes	<code>-O3 -fprofile-use</code>	Absolute fastest
<code>minimal</code>	0	0	No	<code>-O2</code>	Embedded
<code>l4re</code>	0	0	No	<code>-O2</code>	L4Re microkernel
<code>profile</code>	0	0	No	<code>-O1 -g</code>	Profiling

A.1.10 Optimization Level Guidelines

- **-O0 -g (development)** — fastest compile, slowest runtime ($\approx 10\times$ below optimized); full symbols, all assertions. Use for initial development and crash debugging.
- **-O2 (balanced)** — moderate compile overhead; 3–4 $\times$ runtime gain over `-O0`. Default production-test builds.
- **-O3 (high performance)** — aggressive inlining and vectorization; 5–6 $\times$ over `-O0`. Production releases.

- **-O3 -flto -march=native -fprofile-use (maximum)** — multi-stage build; 7–8× over -O0. Release and benchmark artifacts. Use `make pgo`.

A.1.11 Usage Examples

Maximum performance native build

```
1 make clean && make pgo
```

Produces a PGO-optimized binary with assembly optimizations, direct threading, and link-time optimization.

Debug build with profiler

```
1 make clean && make profile
2 ./build/starforth --profile 3 --profile-report
```

Cross-compile for Raspberry Pi 4

```
1 make rpi4-cross
2 scp build/starforth pi@raspberrypi.local:~/
```

Produces a statically linked ARM64 binary optimized for Cortex-A72.

Minimal embedded build

```
1 make minimal
```

Produces a freestanding binary with no libc dependencies.

AMD Zen 3 custom build

```
1 make clean
2 make CFLAGS="$(BASE_CFLAGS) -O3 -march=znver3 \
3     -DUSE_ASM_OPT=1 -DUSE_DIRECT_THREADING=1 -DNDEBUG" \
4     LDFLAGS="-flto -s"
```

Generate assembly listings

```
1 make asm
2 less build/stack_management.s
```

A.1.12 Compiler Support

- **GCC** — recommended. Minimum GCC 7.0; GCC 11.0+ preferred for improved PGO and ARM64 code generation.
- **Clang** — minimum Clang 10.0; computed-goto and all assembly optimizations supported.
- **ARM64 cross-compilation from x86_64:**

```
1 sudo apt-get install gcc-aarch64-linux-gnu
2 make CC=aarch64-linux-gnu-gcc rpi4-cross
```

Static linking (`-static`) is recommended for cross-compiled binaries.

A.1.13 Performance Tuning Checklist

For maximum throughput:

- Use `make pgo` (profile-guided optimization).
- Enable `USE_ASM_OPT=1` and `USE_DIRECT_THREADING=1`.
- Define `NDEBUG`.
- Pass `-O3 -flto -march=native`.
- Add `-fno-plt -fno-semantic-interposition` to reduce indirection.
- Add `-funroll-loops -finline-functions` for code expansion.
- Link statically (`-static`) to eliminate PLT overhead.
- Strip symbols (`-s`) to reduce binary size.

Expected result: $\approx 7\text{--}8\times$ faster than a debug build; $\approx 1.2\text{--}1.5\times$ faster than a plain `-O3` build.

A.1.14 Troubleshooting

“Unknown architecture” warning The build falls back automatically to pure-C implementations. Assembly optimizations are disabled. To add support for a new architecture, extend `arch_detect.h`.

Direct threading shows no performance gain Verify that both `USE_DIRECT_THREADING=1` and `USE_ASM_OPT=1` are set, that the target is `x86_64` or `ARM64`, and that the compiler is GCC or Clang. At runtime, `WORDS-INFO` reports whether direct threading is active.

PGO coverage mismatch warnings Run `make clean && make pgo` to force a full clean two-stage rebuild. Stale coverage data from a prior instrumentation pass causes these warnings.

Appendix B

HOLA Analytics Protocol

B.1 The HOLA Shared-Memory Protocol (Phase 1)

Phase 1 establishes the Host Observation & Logistics Adapter (HOLA) as the stable contract between the StarForth VM and external analyzers. The runtime exposes a fixed-size analytics heap — 10 MiB by default — implemented in `physics_runtime.c`. This section specifies the shared-memory schema and command flow that Phase 1 tooling expects.

B.1.1 Memory Layout

The analytics heap divides into four regions. The header carries single-producer/single-consumer metadata identified by `HOLA_SHARED_MAGIC`; the event ring is a lock-free buffer of physics events; the summary region holds aggregated statistics and Bayesian posteriors; and the scratch region is reserved for governance-approved features.

Region	Offset	Size	Notes
Header	0	<code>sizeof(header_t)</code>	Producer/consumer metadata
Event ring	<code>ring_offset</code>	<code>ring_bytes</code>	Lock-free event ring
Summary	<code>summary_offset</code>	<code>summary_bytes</code>	Stats & posteriors
Scratch	<code>scratch_offset</code>	<code>scratch_bytes</code>	Reserved

Table B.1: HOLA analytics heap regions.

The header mirrors the C structure in `include/physics_runtime.h` and must remain packed and aligned exactly as defined. All multi-byte fields use the VM's native endianness. Consumers must validate `magic`, `version_major`, and `version_minor` before touching the heap.

B.1.2 Event Records

Each ring entry begins with a fixed header:

```
1 struct physics_analytics_event_header {
2     uint32_t channel;           // logical stream id
3     uint16_t payload_bytes;    // raw payload size
4     uint16_t reserved;         // alignment padding (zero)
5     uint64_t timestamp_ns;     // producer timestamp (sf_monotonic_ns)
6 };
```

Payloads are padded to eight bytes. The producer updates `produce_seq` atomically after writing each record; consumers advance `read_offset` and bump `consume_seq` once an event is fully processed. Overflow is signalled by incrementing `dropped_events` while leaving the ring untouched, and consumers must treat the next readable event as the resynchronization point.

B.1.3 Channels

Channel	Purpose
0x00000001	Word execution samples (entropy, latency, dictionary id)
0x00000002	Host snapshot deltas (<code>physics_host_snapshot_t</code>)
0x0000FF00–0x0000FFFF	Governance-reserved control/alert events

Table B.2: Recommended HOLA channels.

Channel 0x00000002 carries a binary `physics_host_snapshot_t` in native endianness. Phase 1 adds Pressure Stall Information averages (`avg10/60/300` scaled by 1000), `/proc/stat` totals (`cpu_total_jiffies`, `cpu_idle_jiffies`) for consumer-side delta calculation, cgroup v2 readings (`cgroup_cpu_usage_us` from `cpu.stat`, `cgroup_memory_current_bytes` from `memory.current`, each zero when unavailable), and a `flags` bitmask (`PHYSICS_HOST_FLAG_*`) indicating which groups were populated. Consumers should honor the flag bits to distinguish kernels that expose PSI, cgroup v2, or `/proc/stat`.

B.1.4 Command Protocol

Commands travel through the summary region; its first 256 bytes form a simple request/response mailbox:

```

1 struct hola_command {
2     uint32_t opcode;           // see table
3     uint32_t arg0;             // optional argument / status
4     uint64_t arg1;             // optional payload offset from heap base
5 };

```

Opcode	Direction	Semantics
HOLA_NOP	Either	No-op; producer clears when idle
HOLA_REQUEST_SNAPSHOT	Consumer → VM	Publish snapshot on channel 0x2
HOLA_RESET_RING	Consumer → VM	VM zeroes cursors when safe
HOLA_STATUS_OK	VM → Consumer	Command completed
HOLA_STATUS_BUSY	VM → Consumer	Deferred; retry next tick
HOLA_STATUS_UNSUPPORTED	VM → Consumer	Opcode not understood

Table B.3: HOLA command opcodes.

The VM polls the mailbox at the end of each observation-window hop and writes responses back into the same structure before the next poll. Phase 1 implements only `HOLA_REQUEST_SNAPSHOT`; the remaining opcodes are reserved.

B.1.5 Synchronization Rules

- **Single producer:** the VM is the sole writer to the ring; multiple producers must funnel through the runtime shim.
- **Single consumer:** Phase 1 assumes exactly one analyzer; multi-consumer support will require an indirection table in the summary region.
- **Memory ordering:** producers write payload, then header, then bump `produce_seq`; consumers read `produce_seq` before pulling events and bump `consume_seq` after advancing `read_offset`.

- **Failure semantics:** when `dropped_events` increases, the consumer treats the last coherent boundary as unknown and resynchronizes from the first complete record after the drop.

B.1.6 Artefacts

The runtime implementation lives in `include/physics_runtime.h` and `src/physics_runtime.c` (`physics_runtime_init`, `physics_host_snapshot`, `physics_analytics_publish_event`). The formal model linkage is in `Physics_StateMachine.thy` and `Physics_Observation.thy`, which capture the observation-window semantics this protocol assumes. Future revisions will document multi-node governance flows and captable wiring for L4Re tasks once Phase 2 moves the analyzer into a dedicated microkernel service.

Appendix C

Physics Design Notes

C.1 Physics Signal Inventory

This inventory enumerates the telemetry the physics-driven scheduler can sense today, drawn from two sources: the L4Re microkernel (through the StarshipOS loader stack) and the in-process StarForth VM. It defines the input stage of the scheduler. The guiding metaphor is an operational amplifier: the positive input ties to real host signals, the negative input ties to VM-internal counters, and a Bayesian inference loop stabilizes the gain. The metaphor is a modeling device for the control architecture, not a literal circuit.

C.1.1 L4Re and Microkernel Signals

Kernel Interface Page (KIP). The KIP exposes a high-resolution monotonic clock (`l4_kip_clock_ns()`) for timing when no RTC is present, a kernel build fingerprint for correlating inference data with kernel revisions, the scheduling granularity (an upper bound on how fine-grained the physics scheduler can react), physical memory descriptors (how much RAM can be earmarked for in-RAM inference buffers), and platform/SMP information. SMP availability doubles as a noise hook: work can be scheduled on sibling cores to inject controlled jitter and keep the model honest.

Scheduler object. `l4_scheduler_info()` reports the online CPU bitmap and maximum CPU count, bounding how many physics loops run in parallel. `l4_sched_param_t` (priority and quantum) records the loader's configured timeslice as a prior per VM. The scheduler-class flags reveal whether fixed-priority or weighted-fair-queue scheduling is active, which shapes how aggressively words may be heated or cooled. `l4_sched_cpu_set()` pins the main or physics worker thread to a core subset during experiments.

Loader and environment capabilities. The loader script (`quark.lua`) wires named capabilities the runtime can tap: `vbus_storage` for block-device enumeration and driver statistics (queue depth, error counts); `ahci_chan` for completion-interrupt rates and per-port error registers; `rtc_ns` for wall-clock sync; `cons_mux` for console input cadence as a proxy for interactive heat; `L4.Env.log` for scheduler and hardware warnings that should bias temperature decay; and `L4.Env.vesa` for framebuffer refresh/blit latency.

Further kernel APIs. `l4_thread_control()` and `l4_thread_ex_regs()` expose per-thread UTCB pointers and instruction pointers for correlating VM stalls with kernel scheduling; `l4_ipc_error()` surfaces IPC failure codes that could feed entropy penalties for fault-prone words; and virtual IRQ devices can be counted to derive environmental noise injected into the VM.

C.1.2 StarForth VM Signals

Core runtime state. `DictEntry.entropy` is the per-word execution counter, the baseline for macro temperature. `DictEntry.physics` holds the tunable knobs (temperature, last-active timestamp, mass). The data and return stack pointers (`VM.dsp`, `VM.rsp`) report stack depth, whose steep excursions imply turbulent execution and feed variance estimates. Mode and state fields reveal interpret-versus-compile state and the currently executing word. The `error` and `halted` flags are binary fault signals that spike entropy decay when they flip. Log volume and severity mix are themselves telemetry: heavy warnings can cool risky words.

Profiler and instrumentation. `WordProfile` supplies per-word total time, call count, and min/avg/max latency — a direct feed for `avg_latency_ns` and Bayesian likelihoods. `MemoryProfile` supplies read/write counts and bytes as a mass-energy proxy for memory-thrashing words. The profiler counters (VM cycles, dictionary lookups, stack ops, allocations) seed default temperatures and heat capacities. `profiler_word_count()` stays available even without detailed profiling, and `vm_debug_dump_state()` provides a structured post-mortem the Bayesian tool can parse to reset priors after a crash.

Block and storage subsystem. Block-subsystem globals report working-set size, dirty-block counts, and block-allocation-map churn. `blkio_info()` exposes device geometry and the read-only bit, informing whether a cooling word should migrate to RAM or disk tiers. `blkio_read/write` return codes give immediate error feedback. Cache slots track hit/miss rate and write-back frequency to infer subsystem momentum.

C.1.3 Op-Amp Signal Flow

The signal flow follows the amplifier metaphor in five stages:

1. **Positive input (microkernel)** — real-world noise: CPU availability, I/O latency, RTC drift, IRQ storms.
2. **Negative input (VM)** — internal state: entropy, latency, stack tension, storage dirty set.
3. **Amplifier** — the Bayesian inference loop adjusts priors and updates word physics.
4. **Output** — updated `DictPhysics` structs and scheduler hints that modulate execution order and block placement.
5. **Feedback** — the observation window width is adjusted by variance (a gauge study) and the cycle repeats.

C.1.4 Messaging and IPC Considerations

The pub/sub backbone is shared-memory-first: a ring buffer with sequence counters inside a dedicated analytics heap (10 MiB by default). Producers write events, flip a counter, and continue, with no blocking semantics inside the VM. On L4Re, where IPC is synchronous, IPC serves only as a notification channel — a publisher pokes a notification thread that drains the ring and forwards to subscribers; virtual IRQs offer an alternative non-blocking wakeup. On Linux the same API is backed by condition variables or eventfd behind a common shim, keeping the VM path identical across platforms. All state stays resident in a fixed-size analytics heap with no dynamic expansion. That heap is separate from the 5 MiB VM arena (`VM_MEMORY_SIZE`), so the dictionary, stacks, and block-subsystem budgets remain untouched.

C.1.5 Host Snapshot Shim and Analytics Heap

Phase 1 ships a concrete implementation. `physics_runtime_init()` reserves the analytics heap (10 MiB default) whose layout the HOLA contract documents, publishing the header and region descriptors HOLA consumes. `physics_host_snapshot()` abstracts POSIX and L4Re scheduler probes and feeds ring-buffer events on channel `0x00000002` via `physics_analytics_publish_event()`. The heap header, event records, and mailbox schema live in `include/physics_runtime.h`; the runtime lives in `src/physics_runtime.c`. The interface is deliberately ABI-stable so governance tooling can mirror it without pulling in C sources. On the POSIX path, Phase 1 additionally captures Linux PSI values mapped into `psi_*_avg{10,60,300}_milli`, `/proc/stat` total and idle jiffies, and `cgroup v2` CPU and memory usage where available; flag bits advertise which sources were populated.

C.1.6 Conventions and Constraints

Several conventions govern the inference loop. The observation window combines a time-based heartbeat with event-count triggers: events act as “excitement” that boosts entropy, while publish and decay operations cool at roughly half that rate (tunable). All computation uses 64-bit fixed-point integers. Physics snapshots can optionally be persisted into FORTH block storage and reloaded during (INIT) to simulate a warm boot or replay a training sequence. Descriptor inheritance flows from module, vocabulary, and VM defaults down to individual words, so sensible priors seed at multiple levels; every tier exposes the same attribute schema — temperature, latency, mass, state flags, ACL hints, pub/sub mask, and pinned flag. VM-level rollups mirror per-word metrics and define operating bands (COLD, WARM, HOT, CRITICAL) that governance and the scheduler shim can respond to. Isabelle captures the formal state machine, invariants, and IPC-handshake proofs; HOLA defines the shared-memory layout and control protocol consumed by both the VM and external analyzers.

A primitive seed table (`physics_metadata_apply_seed()`) installs initial priors for high-impact primitives — control flow, I/O, block subsystem, save-system — so temperature and latency estimates do not start at absolute zero; governance tooling can extend or override the table once the Bayesian loop is in place.

C.1.7 Immediate Research Tasks

1. Prototype a thin KIP/scheduler shim exposing the listed signals to userland C with no filesystem: on L4Re via `l4re_kip()`, on POSIX via a stub that feeds monotonic time and scheduler defaults.
2. Inventory the IO-server (vbus) protocol to pull queue-depth and error counters for AHCI, NVMe, and virtio backends.
3. Define the shared-memory layout between the VM and the Bayesian analyzer, defaulting to a 10 MiB heap (header, ~6 MiB event ring, ~3 MiB summary/scratch, padding), honoring the platform split between POSIX and L4Re messaging.
4. Extend the profiler to snapshot `MemoryProfile` deltas without enabling full verbose mode, keeping overhead low.
5. Derive initial priors for key primitives (control, I/O, block) from handcrafted knowledge plus the loader’s priority and quantum configuration.

